Sudha Gopalakrishnan Brain Centre - BFI-NISSL HD Image Registration

Documentation of Sakiley Pranay Deep's Work

Overview

This document provides a comprehensive workflow for registering Block Face Imaging White (BFIW) images with NISSL-stained histological images at high definition (HD) resolution. The registration process enables precise alignment between different imaging modalities of the same brain tissue, facilitating accurate analysis and visualization.

The complete workflow consists of 8 sequential steps that transform and align images through multiple coordinate systems:

1. **BFIW Cleaning & Stacking** - Prepare and clean Brain Fluorescence Imaging (BFI) images
2. **NISSL Background Removal** - Clean NISSL histological images
3. **Initial Transformation Calculation** - Find transformations between BFI and NISSL stacks for selected slices
4. **Interpolation for Complete Coverage** - Generate transformations for all image pairs (Part A)
5. **Stack-to-Thumbnail Transformation** - Map stacked NISSL to original thumbnail images (Part B)
6. **Thumbnail-to-HD Transformation** - Map thumbnails to high-definition images (Part C)
7. **Transformation Chaining** - Combine all transformation matrices
8. **Live Registration Application** - Deploy interactive visualization tool

---

1 Prerequisites

Before beginning this workflow, ensure you have access to:

- **Raw BFI images** from Sivamani sir's team (preferably pre-stacked)
- **NISSL stack images** (TIFF format) from Sivamani sir's team
- **Original NISSL thumbnail images** stored on DGX servers
- **Large HD NISSL images** for final registration
- **Required software**: ilastik, Python environment with necessary libraries
- **Code repositories**:
  - Main repository: https://github.com/pranaydeep139/brain-registration
  - Colab notebook: https://colab.research.google.com/drive/19gX4xkrcsByyVWG8LIWQy2Elp7n29LwP?usp=sharing

---

2 Detailed Step-by-Step Workflow

2.1 Step 1: BFIW Cleaning and Stacking

Objective: Prepare clean, background-removed, and properly stacked BFI images.
Process:

1. **Obtain Source Images**
   - Acquire stacked BFIW images for the brain sample from Sivamani sir's team (preferred)
   - If stacked images are unavailable, raw BFI images will need to be processed and stacked
2. **Format Conversion**
   - Convert TIFF files to individual image folders using `tiff_to_folder.py`
   - Location: https://github.com/pranaydeep139/brain-registration
   - Adjust script arguments according to your specific file structure
3. **Color Enhancement**
   - Apply Multi-Scale Retinex Color Restoration using `msrcr.py`
   - Location: Pranay's folder on DGX server
   - This step improves image contrast and color balance
4. **Background Removal Using ilastik**
   - **Preparation**: Downsample all color-restored images by 4x for efficient processing
   - **Sample Selection**: Choose approximately 15 diverse images distributed evenly across all slices (spacing: 100-150 slices apart)
   - **Training Process**:
     - Open ilastik and create a new Pixel Classification project
     - Load the 15 selected downsampled images
     - Manually label pixels as either "background" or "foreground" (brain tissue)
     - Train the classification model
   - **Batch Processing**:
     - Apply the trained model to all images using ilastik's batch processing mode
     - Use Simple Segmentation mode to generate foreground masks
   - **Mask Application**: Use generated masks to remove background from original color-restored images
     - Script: `bg_removal.py` (located in Pranay's DGX folder)
     - May require adjustments based on mask format
5. **Alternative Workflow for Unstacked Images**
   - If pre-stacked BFI images are unavailable:
   - Retrieve raw BFI images from DGX servers
   - Perform background removal first (as described above)
   - Apply image stacking using `align.py` or `new_align.py` (located in Pranay's DGX folder)

2.2 Step 2: NISSL Background Removal

Objective: Clean NISSL images by removing background artifacts and standardizing format.

Process:

1. **Format Conversion**
   - Convert NISSL TIFF stack from Sivamani sir's team to individual images
   - Use `tiff_to_folder.py` from https://github.com/pranaydeep139/brain-registration
2. **File Standardization**
   - Rename all NISSL images to standardized format: {`slice_number`}`.jpg`
   - Use `rename_nissl.py` from https://github.com/pranaydeep139/brain-registration
   - Adjust script parameters if needed for your specific naming convention
3. **Sample Selection and Background Removal**
   - Select representative NISSL images with approximately 200-slice spacing
   - Inspect selected images for background characteristics
   - Remove background by replacing high-intensity pixels (typically ¿235 for all RGB channels) with pure white
   - Implementation: Use the background removal code in the Colab notebook: https://colab.research.google.com/drive/19gX4xkrcsByyVWG8LIWQy2Elp7n29LwP?usp=sharing

2.3 Step 3: Find BFI Stack to NISSL Stack Transformations

Objective: Calculate precise geometric transformations between corresponding BFI and NISSL image pairs.

Process:

1. **Setup and Preparation**
   - Use the Colab notebook: https://colab.research.google.com/drive/19gX4xkrcsByyVWG8LIWQy2Elp7n29l
     usp=sharing
   - Select corresponding BFI slices for each NISSL slice chosen in Step 2
2. **Image Preprocessing**
   - Pad both NISSL and BFI images to 5000×5000 pixels using the provided code cell
   - This standardization ensures consistent processing dimensions
3. **Manual Orientation Correction**
   - Visually inspect each NISSL-BFI image pair
   - If significant orientation differences exist, manually rotate the BFI image
   - Use the rotation code provided in the Colab notebook
   - Record the rotation angle applied for each slice
4. **Automated Registration Optimization**
   - Run the optimization algorithm that maximizes Intersection over Union (IoU)
   - The algorithm calculates optimal parameters:
     - **Scale factor**: Relative size adjustment between images
     - **Rotation angle**: Additional rotation needed after manual correction
     - **Translation (tx, ty)**: Horizontal and vertical offset corrections
   - Record all parameters for each processed slice pair

2.4 Step 4: Use Interpolation and Get Transformation for All Pairs - Part A

Objective: Generate transformation matrices for all brain slices using interpolation between calculated control points.

Process:

1. **Data Organization**
   - Ensure all images are stored in: `images_data/{brain-sample-id}/`
   - BFI images: `bfi-{slice_number}.png` format
   - NISSL images: `nissl-{slice_number}.jpg` format
2. **Configuration Setup**
   - Open `export_transforms.py`
   - Locate the `BRAIN_SAMPLES_CONFIG` section
   - Under your current brain sample ID, input the following parameters:

   Required Parameters for Each Slice:

   - `nissl_dims`: Dimensions of NISSL stack images from Sivamani sir's team
   - `bfw_dims`: Dimensions of BFI stacked images
   - `slice_num`: Individual slice number
   - `bfw_css_rotation_degree`: Manual rotation applied in Step 3
   - `scale`: Scale factor from optimization algorithm
   - `angle`: Rotation angle from optimization algorithm
   - `tx`: X-direction translation from optimization algorithm
   - `ty`: Y-direction translation from optimization algorithm
3. **Execution and Output**
   - Run `export_transforms.py`
   - The script performs linear interpolation between known transformation points
   - Generates transformations for all slices, including those not manually processed
   - Output: `all_slice_transformations.json` containing transformation matrices for all brain samples

2.5 Step 5: Find Transformations Between Stacked NISSL Images and Original Thumbnail Images - Part B

Objective: Establish transformation relationships between processed NISSL stack and original thumbnail images.

Process:

1. **Data Preparation**
   - Collect all original NISSL thumbnail images for the brain sample from DGX servers
   - Organize into a dedicated folder
   - Rename files to match format: `slice_number.jpg` (e.g., `340.jpg` for slice 340)
2. **Transformation Calculation**
   - Execute `thumb_nissl.py` with the following inputs:
     - NISSL stack folder (from previous steps)
     - NISSL original thumbnails folder
     - Ensure identical slice numbers have matching filenames in both folders

3. **Output Processing**
    - Raw transformation matrices are generated
    - Clean the output JSON file using `clean_json.py`
    - Final output: {`brain-sample-id`}`_t_to_s.json`
    - This file contains transformation matrices mapping NISSL stack to original thumbnails

2.6 Step 6: Find Transformation Between Original Thumbnails and Large HD Images - Part C

Objective: Calculate the transformation between thumbnail images and high-definition NISSL images.
Process:

1. **Transformation Calculation**
    - Run `convert_to_canvas.py`
    - The script uses original thumbnails as an intermediary coordinate system
    - Processes both NISSL stack and large HD NISSL images through thumbnail space
2. **Required Inputs**
    - Transformation JSON from Step 5 ({`brain-sample-id`}`_t_to_s.json`)
    - Downsampling factor used to create thumbnails from HD images
    - This factor represents the scale reduction from HD to thumbnail resolution
3. **Output Generation**
    - Produces: {`brain-sample-id`}`_c_to_s.json`
    - Contains transformation matrices between large HD NISSL images and NISSL stack

2.7 Step 7: Chain the Transformations

Objective: Combine all transformation matrices to create the complete BFI-to-HD-NISSL registration.
Process:

1. **Input Preparation**
    - Collect all transformation files from previous steps:
        - `all_slice_transformations.json` (from Step 4)
        - {`brain-sample-id`}`_c_to_s.json` (from Step 6)
    - Ensure brain sample ID is correctly specified
2. **Transformation Chaining**
    - Execute `chain_transforms.py`
    - The script mathematically combines:
        - Part A: BFI stack to NISSL stack transformations
        - Part B: NISSL stack to thumbnail transformations
        - Part C: Thumbnail to HD image transformations
3. **Final Output**
    - Generates: {`brain-sample-id`}`_original_to_bfiw.json`
    - Contains complete transformation matrices for BFI-to-Large-HD-NISSL registration
    - This file enables direct mapping between BFI and HD NISSL coordinate systems

2.8 Step 8: Live BFI-NISSL Registration View

Objective: Deploy an interactive web application for real-time visualization of registered images.
Process:

1. **Application Launch**
   - Navigate to the repository: https://github.com/pranaydeep139/brain-registration
   - Run: `python app.py`
   - This starts the Flask web server
2. **Access and Usage**
   - Open web browser and navigate to: `localhost:5000/viewer/{brain_sample_id}/{slice_number}`
   - Replace {`brain_sample_id`} with your actual brain sample identifier
   - Replace {`slice_number`} with the desired slice number
   - The application provides live registration visualization with interactive controls
3. **Features**
   - Real-time overlay of BFI and NISSL images
   - Interactive transformation controls
   - Slice-by-slice navigation
   - Quality assessment tools

---

3 File Organization and Naming Conventions

3.1 Directory Structure for images

```
images_data/
 {brain_sample_id}/
    bfi-{slice_number}.png
    nissl-{slice_number}.jpg
```

3.2 File Naming Standards

- **BFI Images**: `bfi-{slice_number}.png` (e.g., `bfi-340.png`)
- **NISSL Stack Images**: `nissl-{slice_number}.jpg` (e.g., `nissl-340.jpg`)
- **NISSL Thumbnails**: {`slice_number`}`.jpg` (e.g., `340.jpg`)
- **Transformation Files**: {`brain_sample_id`}_{`transformation_type`}`.json`

---

4 Key Scripts and Their Functions

---

5 Important Notes and Considerations

5.1 Quality Control

- Visually inspect each transformation result before proceeding to the next step
- Verify that slice numbering remains consistent across all processing steps
- Check transformation matrix validity by testing on sample image pairs

Script Name	Primary Function	Input Requirements	Output Generated
tiff_to_folder.py	Convert TIFF stacks to individual images	TIFF file path, output directory	Folder of individual image files
msrcr.py	Multi-Scale Retinex Color Restoration	Raw BFI images	Color-enhanced images
bg_removal.py	Apply background masks to remove artifacts	Images and corresponding masks	Background-removed images
align.py / new_align.py	Stack alignment for raw BFI images	Individual BFI images	Aligned image stack
rename_nissl.py	Standardize NISSL image filenames	NISSL image folder	Renamed image files
export_transforms.py	Generate interpolated transformations	Manual transformation parameters	all_slice_transformations.json
thumb_nissl.py	Calculate stack-to-thumbnail transformations	NISSL stack and thumbnail folders	Raw transformation matrices
clean_json.py	Clean and format transformation JSON	Raw transformation JSON	{brain_sample_id}_t_to_s.json
convert_to_canvas.py	Calculate thumbnail-to-HD transformations	Thumbnail transformations, scale factor	{brain_sample_id}_c_to_s.json
chain_transforms.py	Combine all transformation matrices	All transformation JSON files	{brain_sample_id}_original_to_bfiw.jso
app.py	Launch interactive registration viewer	Final transformation matrices	Web application interface

5.2 Troubleshooting

- If ilastik training fails, ensure sufficient diversity in training samples
- For poor registration results, consider adjusting the manual rotation angles in Step 3
- Memory issues may require processing images in smaller batches

5.3 Performance Optimization

- Use appropriate downsampling factors to balance processing speed and accuracy
- Consider parallel processing for large datasets
- Monitor disk space usage during intermediate file generation

5.4 Data Backup

- Maintain backups of original data before processing
- Save intermediate results at each major step
- Document any parameter adjustments made during processing

6 Other works:

- Brain region segmentation, by fine-tuning Medical-SAM2, achieved an IOU of 0.82
- 3d-reconstruction of the brain sample from the stacked and processed BFIW images.

This workflow has been developed and tested by Sakiley Pranay Deep at the Sudha Gopalakrishnan Brain Centre. For technical support or questions regarding implementation, refer to the provided code repositories and documentation.